

Studying S-Boxes and the Feistel Network in DES with the Multilayer Perceptron Model

Hector Li

August 2021

1 Background

Cryptography has been an indispensable part of online security. It permeates every aspect of modern life and its significance can range from protecting credential communications between the military headquarters and the front that might turn the tide of a grueling campaign, to safeguarding an encrypted personal diary from prying eyes from curious siblings.

Two major questions come naturally with any crypto-model. Is the underlying algorithm secure? How widely adopted is this security protocol? Both security goals are intuitive. No one wants his or her password to be broken easily by an attacker, which defeats the purpose of setting up a password system. In addition, based on real-world scenario where a crypto-system is used, different people want different publicity of the protocol. An ideal military protocol would only be known to the headquarter and the army taking the action, whereas a common tested algorithm would help small corporations reduce the cost of handling user information and the government regulate the privacy agreements.

Therefore, the U.S. National Institute of Standards and Technology (NIST) has held multiple rounds of competitions for the data encryption standard. In 2000, NIST chose the block cipher Rijndael as the Advanced Encryption Standard (AES) to replace the Data Encryption Standard (DES), which was standardized in 1977. Even though DES has been criticized for its short 56-bit key length, government and banking still use the Triple-DES implementation of DES.

The concern over the security of a protocol depends on the types of attacks. First, the adversary can have the following goals of descending difficulty [2]:

1. Recover the secret key.
2. Systematically recover plaintext from ciphertext, not necessarily learning the secret key.
3. Learn some partial information about the plaintext from the ciphertext, other than its length.

If the adversary cannot achieve 1 or 2, the crypto-system is *totally secure*. If the adversary cannot achieve 3, the scheme is *semantically secure*.

Generally, the attacks can be categorized in the following way, depending on how the adversary is able to interact with the crypto-system.

- Passive attacks:

- Ciphertext-only attack: The attacker tries to derive information by simply viewing ciphertexts.
- Known-plaintext attacks: The attacker knows some plaintext and the corresponding ciphertext.
- Active attacks:
 - Chosen-plaintext attack: The attacker can choose some plaintext and obtain the corresponding ciphertext.
 - Chosen-ciphertext attack: The attacker has a target ciphertext to decrypt, and can obtain the (plaintext, ciphertext) pair for any other message.

In addition to the attack capability, the security is also dependent on the computational power of the adversary.

- Information-theoretic security: The attacker has infinite computational resources.
- Complexity-theoretic security: The attacker has a ‘polynomial-time Turing machine.’
- Computational security: The attacker is computationally bound by his or her computational resources.

Definition 1 *A crypto-system is secure if it is semantically secure against chosen-plaintext attacks by a computationally bounded adversary.*

With the encryption algorithm, one could generate large samples of the plaintext-ciphertext pairs. One natural question arises, whether one could train the machine with the plaintext-ciphertext samples and uncover the some relation between the ciphertext and the plaintext. Unfortunately, the answer to that question is “no” by the definition of a secure crypto-system. Since a secure crypto-system is semantically secure against chosen-plaintext attacks by a computationally bound adversary, a machine trained with finite plaintext-ciphertext samples would be unable to learn some partial information about the plaintext from the ciphertext other than its length. However, what machine learning could do is to reveal the weakness of an insecure crypto-system. In this paper, we will inspect the design of substitution-boxes (S-boxes) and Feistel network in DES.

Both AES and DES relies on substitution, and the substitution process is achieved via the S-boxes. In particular, S-boxes are the only non-linear part of DES. When IBM first shared the DES algorithms to the U.S. National Security Agency (NSA), NSA modified the S-boxes to make the encryption resistant to the differential cryptanalysis then unknown to the public. However, in 1993, Mitsuru Matsui discovered that DES is much more prone to linear cryptanalysis. Although the attack is more theoretical (requiring around 2^{47} known plaintexts), it highlights a possible weakness in the S-box design.

2 Introduction

2.1 DES Structure

The DES algorithm is designed to encipher and decipher blocks of data of 64 bits with a 64-bit key. The structure of the algorithm can be shown in the figure below.

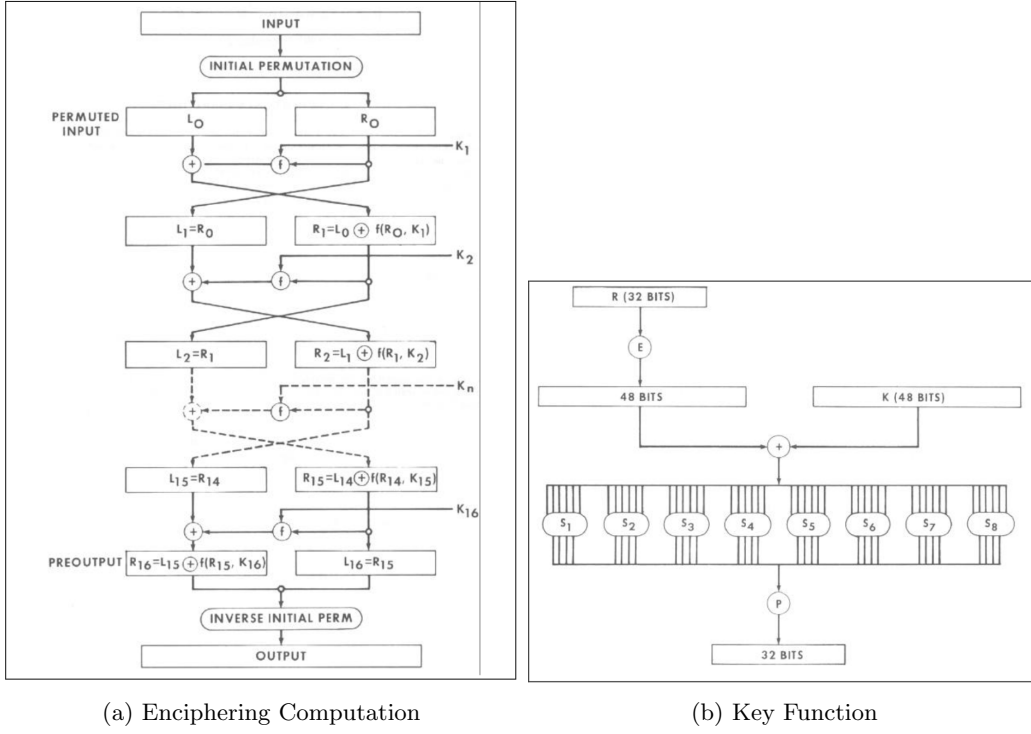


Figure 1: DES Structure

2.2 S-Box design

A S-box is a function that takes in a m -bit integer and returns a n -bit integer. In DES, each S-box takes a 6-bit integer input and returns a 4-bit integer output. Consider the S-box S_1 in the official documentation

$$S_1 = \begin{matrix} & 14 & 4 & 13 & 1 & 2 & 15 & 11 & 8 & 3 & 10 & 6 & 12 & 5 & 9 & 0 & 7 \\ \begin{matrix} 0 \\ 1 \\ 2 \\ 3 \end{matrix} & \begin{matrix} 0 & 15 & 7 & 4 & 14 & 2 & 13 & 1 & 10 & 6 & 12 & 11 & 9 & 5 & 3 & 8 \\ 4 & 1 & 14 & 8 & 13 & 6 & 2 & 11 & 15 & 12 & 9 & 7 & 3 & 10 & 5 & 0 \\ 15 & 12 & 8 & 2 & 4 & 9 & 1 & 7 & 5 & 11 & 3 & 14 & 10 & 0 & 6 & 13 \end{matrix} \end{matrix}.$$

We label the rows 0, 1, 2, 3 from top to bottom and the columns 0, 1, $\dots$, 15 from left to right. For any input $\overline{x_0x_1x_2x_3x_4x_5}$ on base 2, where $x_0, x_1, x_2, x_3, x_4, x_5 = 0, 1$, $S_1(\overline{x_0x_1x_2x_3x_4x_5})$ returns the value on the row $\overline{x_0x_5}$ and column $\overline{x_1x_2x_3x_4}$.

2.3 Feistel Network

The 64-bit input block of plaintext is subjected to the initial permutation, undergoes 16 rounds of Feistel networks and then produces the ciphertext after an inverse initial permutation. Before first round, the block is broken into the left block L_0 , the first 32 bits, and the right block R_0 , the last 32 bits. For any round n , where $n = 1, \dots, 16$, we have $L_n = R_{n-1}$ and $R_n = L_{n-1} + f(R_{n-1}, K_{n-1})$, where f is the round function and K_{n-1} is the key element.

3 S-Box Analysis with the Multilayer Perceptron Model

Let us consider a generalized S-box, a function that takes in a m -bit integer and returns a n -bit integer. By definition, we know that locality does not hold for this function, since each bit of the input is independent and there is no relation between neighboring bits. However, each bit of the output might be related to the sum of input bits with different weights.

Let the input bits be $x_0, \dots, x_{m-1}$ and the output bits be $y_0, \dots, y_{n-1}$, where $x_i, y_j = 0, 1$, and $i = 0, \dots, m-1$ and $j = 0, \dots, n-1$. If we consider $y_0, \dots, y_{n-1}$ as labels, one immediate model that comes up is perceptron for each label.

We consider the most simple S-box implementation which outputs a random n -bit integer for any input. It is easy to see that this model would be a poor estimation. Since the output is randomly generated, each output bit is independent, so it suffices to consider only one label. Since the output is random, if we draw any hyperplane, there will be roughly same number of 0's and 1's on the one side of the hyperplane, so the accuracy would be close to $\frac{1}{2}$.

One remedy to this situation is using the Multilayer Perceptron Model (MLP) instead. By running the Python program, we can see that the training accuracy of the MLP model is 1 when $m < 10$, so it gives a really good estimation. However, as m increases over 10, the training accuracy rapidly decreases and is close to 0.5 when $m = 20$. This decrease of training accuracy is understandable since the input size increases exponentially as m increases, so beyond a certain point, the complexity of the S-box becomes impossible to capture with the weighted sum model inside MLP.

Now we consider only the part where MLP model estimates the S-boxes well, namely when $m < 10$. We can randomly generate labels as the testing set. In the following figure, we show the testing accuracy averaged over

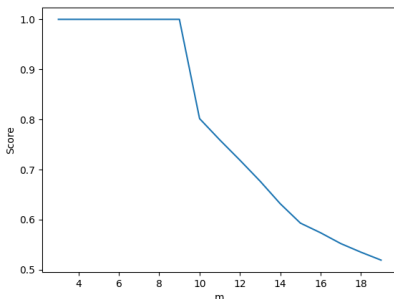


Figure 2: Training Accuracy

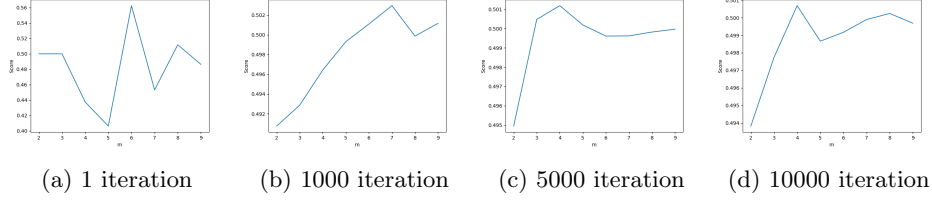


Figure 3: Testing Accuracy

1000, 5000 and 10000 iterations. As we can see, the variance of the graphs is pretty huge. There is no noticeable relation between the testing accuracy and m .

This graph highlights the issue with randomly-defined S-boxes. Even though the test accuracy over high iteration is always close to 0.5, suggesting that the 2 labels are likely independent, the test accuracy is not exactly 0.5. Especially when we do not average the accuracy over many iterations, the test accuracy could deviate pretty far from 0.5. If the model is trained with the input and y_0 label and tested with the input and y_1 label, a testing accuracy away from $\frac{1}{2}$ would allow us to predict y_1 from y_0 , and thus we gain information about the ciphertext other than its length. Therefore, the S-box design with our simple implementation is not semantically secure.

Now we shift our attention to the case where $m = 6$ and $n = 4$, to which the S-box design in DES belongs to. We consider the S-box S_1 in the official documentation

$$S_1 = \begin{matrix} 14 & 4 & 13 & 1 & 2 & 15 & 11 & 8 & 3 & 10 & 6 & 12 & 5 & 9 & 0 & 7 \\ 0 & 15 & 7 & 4 & 14 & 2 & 13 & 1 & 10 & 6 & 12 & 11 & 9 & 5 & 3 & 8 \\ 4 & 1 & 14 & 8 & 13 & 6 & 2 & 11 & 15 & 12 & 9 & 7 & 3 & 10 & 5 & 0 \\ 15 & 12 & 8 & 2 & 4 & 9 & 1 & 7 & 5 & 11 & 3 & 14 & 10 & 0 & 6 & 13 \end{matrix}.$$

We can still try to learn this S-box with the MLP model. By our Python program, we can see that $(X, y_0), (X, y_1), (X, y_2), (X, y_3)$ can all be learned with 100% training accuracy, and the depth of each neural network is 3, so there is only 1 hidden layer. Hence, S_1 can be estimated accurately with a relatively simple neural network. We can use the other labels as the testing set for the neural network. According to our Python program, the testing accuracy for $(X, y_1), (X, y_2), (X, y_3)$ is exactly $\frac{1}{2}$ respectively for the neural network trained with (X, y_0) . Similarly, the testing accuracy for other labels is exactly $\frac{1}{2}$ for neural networks trained with $(X, y_1), (X, y_2), (X, y_3)$. Therefore, we cannot use our MLP model to gain extra information about the ciphertext from the plaintext, so S_1 is semantically secure against the attack based on our MLP model.

Now we can carry out the same analysis on the other S-boxes in the official documentation (they can be found in [1]). By our Python program, they share the same property as S_1 . The neural networks can be trained with an accuracy of 100%, while the testing accuracy is always $\frac{1}{2}$ on other labels. The neural networks also have a depth of 3 each. Following the same argument, we can see that all the S-boxes in the official documentation are semantically secure against the attack based on our MLP model.

However, are those S-boxes the only ones semantically secure against the attack based on our MLP model? We note that each row of S_1 (and S_2 to S_8 in the official documentation) is a

permutation of $0, \dots, 15$. What if we define an S-box where each row is a random permutation of $0, \dots, 15$? By running our Python program over several such randomly generated S-boxes, we find that they all share the same properties as S_1 (including the fact that the depth of the neural network is 3). Therefore, we have the following conjecture that I am unfortunately unable to prove.

Conjecture 1 *For any S-box S_x with $m = 6$ and $n = 4$ designed in the same way as in the official DES documentation except that each row is a random permutation of $0, \dots, 15$, S_x has the following properties:*

- S_x can be learned with an MLP model with 100% training accuracy.
- The neural network of S_x has a depth of 3.
- S_x is semantically secure against MLP-based attack described in this paper.

Does this conjecture work for higher m ? Specifically, will it work for the cases where $m > 10$ where MLP model does not even train well? We can consider the case where $m = 10$ and $n = 4$. Let the first 6 bits of the input denote the index of the row and the last 4 bits denote the index of the column. We construct a $2^6 \times 2^4$ S-box S_r where each row is a random permutation of $0, \dots, 15$. By our Python program, for one specific random S-box generated in this way, the training accuracy of each label is 0.832, 0.856, 0.813, 0.820 respectively, and the testing accuracy is not exactly $\frac{1}{2}$ on other labels, although the depth of the neural network is still 3. Therefore, the conjecture does not hold for $m = 10$ and $n = 4$. We note that the training accuracy of this S-box is close to the training accuracy in the Figure 2 at $m = 10$. We have another conjecture.

Conjecture 2 *The training accuracy of the MLP model on S-boxes is only dependent on m .*

4 Feistel Network Analysis with the Multilayer Perceptron Model

Let us consider a simplified version of the Feistel network in DES that still captures the main idea of the structure. The input of each round of our Feistel network is an 8-bit integer and a 5-bit key K , and the output is also an 8-bit integer. Before the first round, we split the initial input into L_0 , the first 4 bits, and R_0 , the last 4 bits. In the i -th round, the input is $L_{i-1}||R_{i-1}$ ($||$ means concatenation), and the output is $L_i||R_i$, where $L_i = R_{i-1}$ and $R_i = L_{i-1} \oplus f(R_{i-1}, K)$. We define $f(R_{i-1}, K) = S_1(E(R_{i-1}) \oplus K)$, and the expansion function E such that $E(\overline{x_0x_1x_2x_3}) = \overline{x_3x_0x_1x_2x_3x_2}$ where $x_1, \dots, x_3 = 0, 1$.

Similar to our discussion of the S-boxes, since the bits of the output are independent, we can treat each bit of the output as a label and develop an MLP model to estimate the Feistel Network. Let the input be $\overline{x_0x_1 \cdots x_7}$ and the output be $\overline{y_0y_1 \cdots y_7}$, where $x_0, \dots, x_7, y_0, \dots, y_7 = 0, 1$. First, we can choose a specific key $K = 17$ and set the number of rounds to 1. By our Python program, we can see that $(X, y_0), \dots, (X, y_7)$ can all be learned by the MLP model with 100% training accuracy. When we use the MLP model to test other labels, we always get exactly $\frac{1}{2}$ test accuracy. Similar to our discussion of S-boxes, we can conclude that the single round Feistel network can be perfectly estimated by the MLP model and is semantically secure against the attacks based on this MLP model. The depth of this neural network is 3. This result meets our expectation since this Feistel network is by definition an S-box with $m = 8$ and $n = 8$. In fact, this result holds for any choice

of K at round 1. According to our key search Python program, if the number of rounds is 1, all Feistel networks can be learned with 100% training accuracy regardless of the choice K , and the testing accuracy is always $\frac{1}{2}$ on other labels.

However, when we choose $K = 17$ and set the number of rounds to 2, by our Python program, all X, y pairs can be learned with 100% training accuracy except (X, y_4) , which has training accuracy 0.977. The testing accuracy of all neural networks except the one learned from (X, y_4) on other labels is exactly $\frac{1}{2}$, whereas the neural network learned from (X, y_4) gives testing accuracy close to but not exactly $\frac{1}{2}$ on other labels. As the number of rounds increases, we observe more deviation of the training accuracy from 100%, while the accuracy is always above 0.95. This result is still close to the training accuracy of S-boxes shown in Figure 2. However, since the training accuracy deviates as the number of rounds increases, we need to make a slight modification to the Conjecture 2. There might be some terms of the training accuracy that goes to 0 asymptotically, and a good estimate of the training accuracy only depends on m .

Now what if we choose an imperfect S-box to replace S_1 in our Feistel network? We first consider the S-boxes where each row is random permutation of $0, \dots, 15$. By our python program, when the number of rounds is 1 and $K = 17$, the MLP networks exhibit exactly the same behavior as the neural networks developed from the Feistel network with S_1 . The training accuracy also starts to deviate from 100% as the number of rounds increases. This result further shows that MLP models cannot distinguish between the two kinds of S-boxes. If replace S_1 with the S-box that outputs a random 4-bit integer, when the number of rounds is 1 and $K = 17$, we observe that the MLP networks also exhibit the same behavior as the neural networks developed from the Feistel network with S_1 . The training accuracy also starts to deviate from 100% as the number of rounds increases. This result shows that MLP model cannot distinguish those S-boxes in the Feistel network either. Hence, the correlation between output bits in S-boxes do not propagate inside the Feistel network.

5 Conclusion

By our previous discussion, we can see that MLP offers a good (perfect) estimation for S-boxes when $m < 10$ and for Feistel network when the number of rounds is 1. However, as m increases the MLP model very rapidly becomes unusable, while as the number of round increases, MLP is still a decent estimate for the Feistel network. MLP networks can be used to check the semantic security of encryption methods, but it is not sufficient for designing a workable S-box. One future direction of this research is the two conjectures in the S-box section. We can also try to explore the linear relations between input bits by constructing MLP with integer modulo 2 weights.

References

- [1] National Institute of Standards and Technology, “Data Encryption Standard (DES); 1999
- [2] Douglas Stebila, CO 687 Course Notes; 2020